

How to fill the registries — a guide for teams

DocExtract has three **registries** where your team supplies the knowledge that makes the pipeline work well for *your* documents. This guide explains what each one is, what to put in it, and exactly how to add an entry.

You do **not** edit any code. Each registry is a table you add rows to, using a simple registration call (shown for each). Add a row and the pipeline picks it up on the next run — no redeploy.

A rule that applies to all three: entries are *appended*, never overwritten. If you register the same thing twice, the **latest one wins**, and the old one stays in the history. So it's safe to correct a mistake — just register the right version again.

Quick reference — which registry do I need?

I want to...	Use this registry
Tell the AI <i>how to read</i> my documents (give it examples)	Prompt registry
Tell the pipeline <i>what the correct answer is</i> , so it can check itself	Eval registry
Tell the pipeline that <i>two names mean the same thing</i>	Dictionary registry

1. Prompt registry — teach the AI how to read your documents *and* answer questions

What it's for

The AI works better when it's shown a few examples of *your* documents. The prompt registry stores those examples, keyed to your team and report type. It covers **two jobs**, set by the `kind` field:

- `kind="extraction"` — examples of what to **pull out** of a document.
- `kind="query"` — examples of how the **supervisor chat** should **answer questions** for your team.

Think of it as briefing a new analyst: "here's what our documents look like and what to extract" (extraction), and "here's how to answer a supervisor's question well" (query).

You get sensible defaults out of the box. Even before you register anything, both extraction and the supervisor chat run on built-in default instructions *and* worked examples. Registering your own **replaces** the default for your team — so this improves quality, it's never a blocker.

What to fill in

Field	What it means	Example
<code>team</code>	Your team / domain	<code>finance</code>
<code>report_type</code>	The kind of document	<code>primary_report</code>
<code>kind</code>	<code>extraction</code> (pulling values out) or <code>query</code> (answering chat questions)	<code>extraction</code>
<code>preamble</code>	A short instruction in your own words	<code>"Extract each reported ratio with its value and period."</code>
<code>fewshots</code>	Input→output examples — as many as you like (see below)	see example

Few-shot examples are the heart of it. Each is a pair:

- `input_text` — a question or document snippet
- `output_text` — the answer or extraction you want back

You can register **any number** of few-shots — add as many as help. There's no limit, and more good examples generally means better results.

How to add extraction examples

```
python
from docextract.shared.prompt_registry import PromptEntry, FewShot, register_prompt

register_prompt(
    PromptEntry(
        team="finance",
        report_type="primary_report",
        kind="extraction",
        preamble="Extract each reported ratio with its value and reporting period.",
        fewshots=[
            FewShot(input_text="Primary ratio was 15.6% as at 31 March 2025.",
                    output_text='{"metric": "primary ratio", "value": "15.6", "period": "31 March 2025"}'),
            FewShot(input_text="Coverage ratio stood at 120%.",
                    output_text='{"metric": "coverage ratio", "value": "120%"}'),
            # ...add as many as you like
        ],
    ),
    updated_by="your.name@yourorg.example",
)
```

How to add supervisor-chat (query) examples

Same call, `(kind="query")`. Here the examples show how a *good answer* to a supervisor's question should read — grounded, cited, and honest about uncertainty:

```
python
register_prompt(
    PromptEntry(
        team="finance",
        report_type="primary_report",
        kind="query",
        preamble="Answer supervisor questions using only retrieved data; cite",
        fewshots=[
            FewShot(input_text="What is the primary ratio for ACME_CORP?",
                    output_text="The primary ratio for ACME_CORP is 15.6% as",
            FewShot(input_text="What was the peak exposure in the chart?",
                    output_text="The chart shows a peak of about 750. This wa",
            # ...add as many as you like
        ],
    ),
    updated_by="your.name@yourorg.example",
)
```

The supervisor chat automatically pulls **your team's** query examples when answering questions scoped to your team — so the more you register, the more the chat sounds like your team's own analyst.

Tips

- **Good examples beat many mediocre ones** — but there's no cap, so add every example that captures a real pattern or tricky case.
- **Register both kinds.** `(extraction)` improves what's pulled from documents; `(query)` improves how the supervisor chat answers. They're separate — filling one doesn't fill the other.
- **Show the hard cases.** The examples that help most are the formats or questions that would trip up a new analyst.
- **The defaults already flag chart-read values as unverified and refuse to invent figures** — keep that behaviour in your own examples so the chat stays honest.

2. Eval registry — tell the pipeline what "correct" looks like

What it's for

This is how the pipeline **checks its own accuracy**. You register values you *know* are right for specific documents; the pipeline then compares what it extracted against your known-correct answers and reports how often it agrees.

Crucially, this is the *independent* answer key — it's what stops the pipeline from "marking its own homework." You don't need to register everything; even a handful of known values per report type gives a real accuracy signal.

What to fill in

Field	What it means	Example
<code>team</code>	Your team / domain	<code>finance</code>
<code>report_type</code>	The kind of document	<code>primary_report</code>
<code>entity_ref</code>	Which entity the value belongs to	<code>ACME_CORP</code>
<code>metric</code>	The canonical metric name	<code>primary_ratio</code>
<code>expected_value</code>	The correct value (as text)	<code>"15.6"</code>
<code>match</code>	<code>numeric</code> (compare with tolerance) or <code>exact</code> (exact text)	<code>numeric</code>
<code>tolerance</code>	For numeric: how close counts as correct	<code>0.05</code> (± 0.05)
<code>document_id</code>	(optional) pin to one specific document	<code>"acme_q1_2025.pdf"</code>

How to add one

```
python
from docextract.shared.eval_registry import ExpectedValue, register_expected

register_expected(
    ExpectedValue(
        team="finance",
        report_type="primary_report",
        entity_ref="ACME_CORP",
        metric="primary_ratio",
        expected_value="15.6",
        match="numeric",
        tolerance=0.05,          # 15.55-15.65 all count as correct
    ),
    updated_by="your.name@yourorg.example",
)
```

Tips

- Use `numeric` + a **tolerance for numbers** (ratios, amounts) so tiny rounding differences don't count as errors. Use `exact` for things that must match character-for-character (a reference code, a date string).

- **Set the tolerance to what actually matters.** A capital ratio might need ± 0.05 ; a rounded headline figure might tolerate ± 1 .
- **Register the values you most care about being right.** These are the ones the pipeline's accuracy score will be measured on, so choose the high-stakes ones.
- Leave `document_id` blank to apply the expected value across that entity's documents; set it to check one specific file.

3. Dictionary registry — tell the pipeline that two names mean the same thing

What it's for

Your documents call the same thing different names — "CET1 ratio", "Common Equity Tier 1", "core equity ratio" might all mean one metric. The dictionary maps every way a term is written to one **canonical** name, so the pipeline can group and cross-check them.

You don't have to map everything — an unmapped term still flows through and gets extracted. Mapping matters when you want two differently-named values to be **recognised as the same thing** (so a contradiction between them can be caught).

What to fill in

Field	What it means	Example
<code>kind</code>	<code>metric</code> (a measure) or <code>doctype</code> (a document type)	<code>metric</code>
<code>synonym</code>	The term as written in documents	"Common Equity Tier 1"
<code>canonical</code>	The one standard name it maps to	<code>primary_ratio</code>

How to add one

```
python
```

```
from docextract.shared.dictionary_registry import TermMapping, register_term

# map a metric synonym
register_term(
    TermMapping(kind="metric",
                synonym="Common Equity Tier 1",
                canonical="primary_ratio"),
    updated_by="your.name@yourorg.example",
)

# map a document-type synonym
register_term(
    TermMapping(kind="doctype",
                synonym="Annual Filing",
                canonical="annual_report"),
    updated_by="your.name@yourorg.example",
)
```

Tips

- **The `synonym` is matched case- and spacing-insensitively** — you don't need to register every capitalisation. "Common Equity Tier 1", "common equity tier 1", and "COMMON EQUITY TIER 1" all resolve the same.
- **Pick one `canonical` name per real-world thing and stick to it.** All the synonyms for one metric must point to the *same* canonical key, or they won't be grouped together.
- **Map the terms that matter for cross-checking**, not every word. If you never need two names reconciled against each other, mapping them is optional.

Putting it together — a worked example

Say your team `finance` handles `primary_report` documents for `ACME_CORP`, and the documents say "Common Equity Tier 1 was 15.6%".

1. **Dictionary** — so "Common Equity Tier 1" is recognised as your standard metric:

```
python

register_term(TermMapping(kind="metric",
                          synonym="Common Equity Tier 1", canonical="primary_ratio"),
              updated_by="you@org")
```

2. **Prompt** — so the AI knows to pull ratios like this out:

```
python
```

```
register_prompt(PromptEntry(team="finance", report_type="primary_report",
    kind="extraction", preamble="Extract each reported ratio.",
    fewshots=[FewShot(input_text="Common Equity Tier 1 was 15.6%",
        output_text='{"metric":"Common Equity Tier 1","value":15.6}'),
    ],
    updated_by="you@org"))
```

3. **Eval** — so the pipeline can check it got 15.6 right:

```
python

register_expected(ExpectedValue(team="finance", report_type="primary_report",
    entity_ref="ACME_CORP", metric="primary_ratio",
    expected_value="15.6", match="numeric", tolerance=0.05),
    updated_by="you@org")
```

Now the pipeline reads your documents with your examples, recognises your metric's many names, and can measure whether it extracted the value correctly.

Frequently asked

Do I have to fill all three? No. Each works independently. Prompts improve extraction quality; eval measures accuracy; dictionary enables cross-checking. Start with whichever gives you the most value — usually prompts.

What if I make a mistake? Just register the correct version again — the latest entry wins, and the pipeline uses it on the next run. Nothing is overwritten, so there's a full history of who changed what.

Do I need to redeploy anything? No. Registries are data. Add a row, and the next pipeline run uses it.

Who should `updated_by` be? Your name or email — it's the audit trail of who registered what. Use a real identifier, not "system".

What happens if I register nothing? The pipeline still runs. Both extraction and the supervisor chat fall back to built-in defaults that include *worked examples*, not just instructions — so the chat already answers sensibly, flags chart-read values as unverified, and won't invent figures. Unmapped dictionary terms flow through ungrouped; eval simply doesn't score the metrics you haven't provided answers for. Filling the registries makes it *better for your domain* — it's not a prerequisite to run.